

第 1 章 评价类模型

1.1 TOPSIS

定义 1.1 (TOPSIS)

TOPSIS 法，全称为“逼近理想解排序法”，是多属性决策分析中一种常用且有效的技术。基于数据，构建出一个虚拟的“最优方案”（即正理想解和一个虚拟的“最劣方案”（即负理想解）。然后，通过计算每一个评价对象与这两个虚拟方案的相对接近程度，来进行排序。一个好的方案，应该距离最优方案最近，同时距离最劣方案最远。



1.1.1 指标正向化

指标名称	指标特点	例子
极大型（效益型）指标	越大（多）越好	成绩、GDP 增速、企业利润
极小型（成本型）指标	越小（少）越好	费用、坏品率、污染程度
中间型指标	越接近某个值越好	水质评估时的 PH 值
区间型指标	落在某个区间最好	体温、水中植物性营养物量

定理 1.1 (指标正向化)

1. 极小型指标

$$\max -x$$

如果所有的元素均为正数，那么也可以使用

$$\frac{1}{x}$$

2. 中间型指标： $\{x_i\}$ 是一组中间型指标序列，且最佳的数值为 x_{best} ，那么正向化的公式如下：

$$M = \max \{|x_i - x_{best}|\}, \quad \tilde{x}_i = 1 - \frac{|x_i - x_{best}|}{M}$$

3. 区间型指标： $\{x_i\}$ 是一组区间型指标序列，且最佳的区间为 $[a, b]$ ，那么正向化的公式如下：

$$M = \max \{a - \min \{x_i\}, \max \{x_i\} - b\}, \quad \tilde{x}_i = \begin{cases} 1 - \frac{a - x_i}{M} & , x_i < a \\ 1 & , a \leq x_i \leq b \\ 1 - \frac{x_i - b}{M} & , x_i > b \end{cases}$$



```
def min2max(datas: list, offset=1e-9) -> list:
    def normalization(data) -> float:
        return 1 / (data + offset)

    return list(map(normalization, datas))

# 最佳范围被认为是中点
def middle2max(datas, x_min, x_max):
    def normalization(data):
        if data <= x_min or data >= x_max:
            return 0
        elif data > x_min and data < (x_min + x_max) / 2:
            return 2 * (data - x_min) / (x_max - x_min)
        elif data < x_max and data >= (x_min + x_max) / 2:
            return 2 * (x_max - data) / (x_max - x_min)

    return list(map(normalization, datas))

def interval2max(datas, x_min, x_max, x_tolerant_min, x_tolerant_max):
    def normalization(data):
```

```

23 if data >= x_min and data <= x_max:
24     return 1
25 elif data <= x_tolerant_min or data >= x_tolerant_max:
26     return 0
27 elif data > x_max and data < x_tolerant_max:
28     return 1 - (data - x_max) / (x_tolerant_max - x_max)
29 elif data < x_min and data > x_tolerant_min:
30     return 1 - (x_min - data) / (x_min - x_tolerant_min)
31
32 return list(map(normalization, datas))

```

1.1.2 计算过程

考虑边际效应，在某些正向化数值较大的情况下，再增加相同数值所带来的感知提升就越小，可据此将正向标准化的线性假设（Linear Assumption）转化为对数假设（Logarithmic Hypothesis），则这两个参数的式子设置为：

$$R_{ij} = \ln(X_{ij} + 1) \quad (1.1)$$

$X_{ij} + 1$ 是为了防止 $X_{ij} = 0$ 时对数函数无意义。

指标正向化后，使用 TOPSIS-熵权法来确定这些参数的权重。我们构建 m 个事物 n 个评价指标的判断矩阵 $R_{ij}(i, j \in \mathbb{Z}^+)$ ，并将判断矩阵进行归一化处理，得到新的归一化判断矩阵 B ，矩阵 B 的元素获取公式为：

$$B_{ij} = \frac{R_{ij} - \min_{1 \leq k \leq m} \{R_{kj}\}}{\max_{1 \leq k \leq m} \{R_{kj}\} - \min_{1 \leq k \leq m} \{R_{kj}\}} \quad (1.2)$$

计算第 i 个对象在第 j 个指标下的值占该指标总值的比例 p_{ij} ：

$$p_{ij} = \frac{x_{ij}}{\sum_{k=1}^m x_{kj}} \quad (1.3)$$

这样得到了比例矩阵 $\mathbf{P} = (p_{ij})_{m \times n}$ 。而第 j 个指标的信息熵 e_j 计算公式如下：

$$e_j = -k \sum_{i=1}^m (p_{ij} \ln(p_{ij})) \quad (1.4)$$

$k = \frac{1}{\ln(m)}$ 为调节系数，注意在 $m = 1$ 时需要特殊处理（如返回均等权重）。

接下来，将各指标的差异度进行归一化，得到最终的熵权 w_j ：

$$w_j = \frac{1 - e_j}{\sum_{i=1}^n (1 - e_i)} \quad (1.5)$$

熵权得到后，我们采用 TOPSIS - 熵权法计算综合得分，首要步骤为确定最优解向量 Z^+ 和最劣解向量 Z^- 。 Z^+ 是归一化判断矩阵 B 中每列的最大值组成的向量； Z^- 是归一化判断矩阵 B 中每列的最小值组成的向量。其计算公式分别为：

$$Z^+ = \left(\max_{1 \leq i \leq m} \{B_{i1}\}, \max_{1 \leq i \leq m} \{B_{i2}\}, \dots, \max_{1 \leq i \leq m} \{B_{in}\} \right) \quad (1.6)$$

$$Z^- = \left(\min_{1 \leq i \leq m} \{B_{i1}\}, \min_{1 \leq i \leq m} \{B_{i2}\}, \dots, \min_{1 \leq i \leq m} \{B_{in}\} \right) \quad (1.7)$$

在熵权法中，使用的权重 w_j 进行加权处理。第 i 个方案到最优解向量 Z^+ 的距离 D_i^+ 和到最劣解向量 Z^- 的距离 D_i^- 的公式分别为：

$$D_i^+ = \sqrt{\sum_{j=1}^n w_j (B_{ij} - Z_j^+)^2}, \quad D_i^- = \sqrt{\sum_{j=1}^n w_j (B_{ij} - Z_j^-)^2} \quad (1.8)$$

最终只需相对接近度 C_i （第 i 个方案与最优解的相对接近程度），即可视为综合得分，其计算公式为：

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-} \quad (1.9)$$

这些流程的代码如下所示：

```

1 def batch_normalize_from_rules(df, rules: list) -> pd.DataFrame:
2     """读取规则列表，批量处理数据 DataFrame"""
3     for rule in rules:
4         column_name = rule['column']
5         datas = df[column_name].tolist()
6
7         type_to_func = {'max': max2max, 'min': min2max, 'middle': middle2max, 'interval': interval2max}
8         func_to_call = type_to_func.get(rule['type'])

```

```

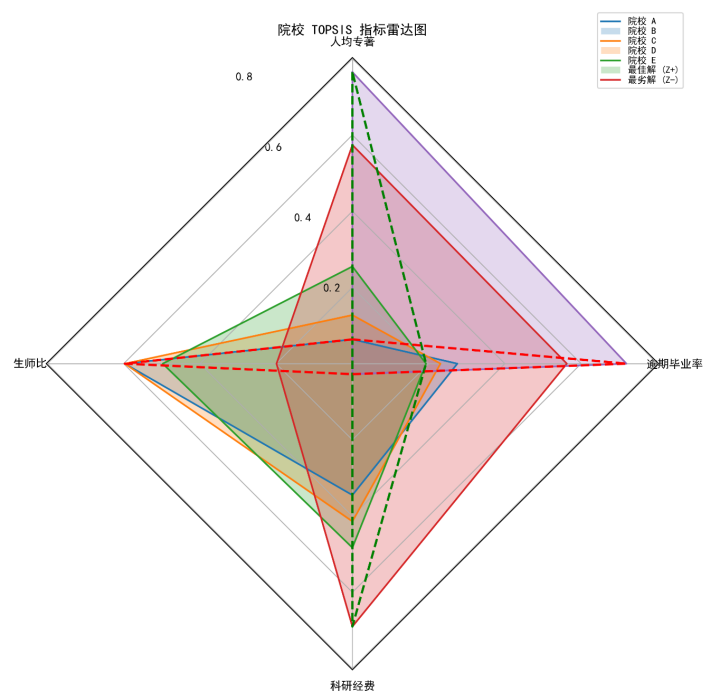
9
10     normalized_values = func_to_call(datas, **rule['params'])
11     df[column_name] = normalized_values
12
13     return df
14
15
16 def normalization(df: pd.DataFrame) -> pd.DataFrame:
17     return df / np.sqrt((df ** 2).sum())
18
19
20 def entropyWeight(df):
21     """计算第 j 个指标下第 i 个样本所占的比重"""
22     df = np.array(df)
23     P = df / df.sum(axis=0)
24     E = np.nansum(-P * np.log(P) / np.log(len(df)), axis=0)
25     d = (1 - E)
26     W = d / d.sum()
27     return W
28
29
30 def topsis(data, weight):
31     # 最优最劣方案 (Z+ 和 Z-)
32     Z = pd.DataFrame([data.max(), data.min()], index=['Z+', 'Z-'])
33     weight = entropyWeight(data) if weight is None else np.array(weight)
34     result = data.copy()
35     result['Z+'] = np.sqrt(((data - Z.loc['Z+']) ** 2 * weight).sum(axis=1)) # 评价对象与最大值的距离
36     result['Z-'] = np.sqrt(((data - Z.loc['Z-']) ** 2 * weight).sum(axis=1))
37     result['综合得分指数'] = result['Z-'] / (result['Z-'] + result['Z+'])
38     result['排序'] = result.rank(ascending=False)['综合得分指数']
39
40     return result, Z, weight

```

TOPSIS 的数据呈现可以用绘图，使用雷达图来呈现。

💡 Tips

https://matplotlib.org/stable/gallery/specialty_plots/radar_chart.html



1.1.3 优点

- 原理直观: 几何意义清晰, 易于理解。
- 计算简便: 算法流程清晰, 易于编程实现。
- 适用性广: 对样本量和指标数量没有严格限制, 对数据分布也无特殊要求。
- 结果合理: 能够充分利用原始数据信息, 得出的排序结果较为合理。

1.1.4 缺点

- 权重依赖性: 指标权重的确定带有一定的主观性, 不同的权重设置可能导致不同的结果。
- 排序的相对性: 当新增或删除方案时, 可能会导致各方案的排序发生变化 (秩逆转)。
- 规范化方法影响: 不同的数据规范化方法可能会对最终结果产生影响。